ASSIGNMENT 8 • COMPREHENSIVE TECHNICAL REPORT

OPTIMIZE ANYTHING

The 14-Day Vibe-Coding Challenge: 10-Track Optimization Portfolio

Discover a process. Formulate the problem. Apply optimization. Prove the improvement.

Course: Optimization Techniques	**Submission Date:** October 2026	**Total Verification:** 151/151 Passing Tests (100%)
Repository: c:\dev\ot-8	**Vibe Coding Tools:** Antigravity & Claude	**Scope:** Compulsory Tracks A through J

Executive Summary: This technical report presents the complete empirical results, mathematical formulations, and code audits for the 14-day vibe-coding optimization challenge. Spanning ten diverse engineering domains—continuous machine learning curvature, discrete knapsack and QUBO Ising formulations, variational quantum algorithms (QAOA, VQE), submodular greedy and multi-core parallel scaling, commercial EV fleet smart charging, mini-compiler optimization passes with ILP register allocation, xv6 kernel stride scheduling and buffer cache profiling, DBMS TPC-H physical design and join-order dynamic programming, canonical quantum algorithms vs classical linear search, and software engineering test suite minimization—this project proves tangible, verified improvements across every track. All AI-assisted development was conducted under strict audit protocols, with 151 independent unit tests enforcing correctness, output equivalence, and zero defect regressions.

1. Chapter 6 Syllabus Traceability Matrix

Per Section 5 of the assignment specifications, every algorithm, mathematical technique, and experiment taught in Chapter 6 has been fully implemented, rigorously verified by automated tests, and benchmarked against empirical baselines:

Chapter 6 Topic	Required Syllabus Algorithms / Techniques	Implementation & Tests	Empirical Outcome / Evidence
ML Foundations	Continuous vs discrete; convex, non-convex, saddle points; loss & regularization; role of optimization in QML	track_A_continuous/ track_I_quantum_algorithms/	Rosenbrock valley, saddle escape, and Hilbert-space quantum kernel mapping.
Continuous Optimization	GD, SGD, Mini-batch, Momentum, NAG, AdaGrad, RMSProp, Adam; Newton, Gauss-Newton; Fisher, Diagonal, Low-Rank, HVP; Backpropagation	first_order_optimizers.py second_order_methods.py mlp_backpropagation.py	Momentum achieved 500x lower loss on Rosenbrock than GD; Newton converged in 1 step; analytical gradient error < 8.9e-11.
Discrete Optimization	Greedy heuristics, 0-1 ILP (HiGHS), Branch-and-Bound, Dynamic Programming; QUBO & Ising spin formulations, penalty terms	knapsack_solvers.py qubo_ising.py	ILP, DP, and B&B; achieved exact global optimum (V = 280); Simulated Annealing hit 100% ground state on Max-Cut.
Quantum Optimization	QAOA (p = 1..3), VQE on Ising Hamiltonian, PQC & QNN, parameter-shift rule, SPSA, QNG; finite-shot noise & barren plateaus	qaoa_maxcut.py vqe_ising.py quantum_gradients.py	QAOA ratio improved from 0.78 (p=1) to 0.93 (p=3); variance scaled as 1/sqrtM; barren plateaus exhibited Var proportional to 2^{-cN}.
Submodular & Parallel	Diminishing returns, lazy greedy (Minoux), (1 - 1/e) bound; Amdahl/Gustafson speedup, work stealing; FedAvg, MGDA, GradNorm	submodular_maximization.py parallel_optimization.py federated_multitask.py	Lazy greedy cut evaluations by 73.3%; BLAS parallel reached 2.84x on 4 cores; FedAvg converged within 1.2% with 5x fewer rounds.
Systems & Domain Tracks	Mini-compiler (4 passes + ILP register allocation), xv6 OS (Stride scheduler & bcache), DBMS (Knapsack index selection & Join DP), QML & Grover, Test Suite Minimization	track_E_case_study/ track_F_compiler/ track_G_xv6/ track_H_dbms/ track_I_quantum/ track_J_swe/	EV charging cost -67.0%; Compiler instructions -45%; xv6 ticks 2.01:1.00; DBMS latency 15.9x faster; QSVM 85.7% acc; SWE test suite -33.8%.

2. Master Strategy Selection Table (Section 4 Workflow)

The following master table outlines the problem characteristics, decision variables, constraints, optimization methods, unoptimized baselines, and measured empirical gains across all ten compulsory tracks:

Track & Domain	Decision Variables	Constraints	Chapter 6 Method	Baseline	Measured Gain / Result
Track A Continuous ML	Weights theta in R^d	Unconstrained / Smooth	Momentum, NAG, Adam, Newton, Gauss-Newton, Fisher/Low-rank	Vanilla Batch GD	Momentum achieved **500x lower loss** on Rosenbrock valley; Newton converged in 1 step.
Track B Discrete & QUBO	Binary x in {0, 1}^n, Spin s_i in {-1, +1}	Weight budget sum w_i x_i <= W, Cardinality K	Greedy ratio, DP (O(NW)), Branch-and-Bound, 0-1 ILP (HiGHS), Simulated Annealing	Greedy heuristic & Random baseline	ILP, DP, and B&B; found identical optimum (V = 280); SA hit 100% ground state on Max-Cut.
Track C Quantum Opt	Angles gamma, beta in [0, 2pi)^p, ansatz weights theta	Unitary gate bounds, parameter limits	QAOA (p=1..3), VQE RealAmplitudes, Param-Shift, SPSA, QNG	Brute Force & Classical Simulated Annealing	QAOA ratio improved 0.78 -> 0.93; VQE ground state error 0.0000%; shot noise proportional to 1/sqrtM.
Track D Submodular / Parallel	Subset S subset_eq V, task allocations, client weights	Budget |S| <= k, worker synchronization	Lazy Greedy (Minoux), Ring All-Reduce, 1F1B, FedAvg, MGDA, GradNorm	Brute force, Single-core, Centralized, Uniform weights	Lazy Greedy saved **73.3% evaluations**; Parallel BLAS reached **2.84x on 4 cores**; FedAvg within 1.2%.
Track E (Case Study) EV Fleet Smart Charging	Charging power P_i,_t in [0, P_max], status u_i,_t	Vehicle energy demands E_i, Transformer peak 120 kW, availability	Continuous Convex (SLSQP), 0-1 ILP (HiGHS), Priority Valley-Filling	Uncontrolled ASAP Charging (170 kW peak, $1,048.50)	Electricity costs cut by **67.0%** ($1,048.50 -> $346.00); peak dropped to 65 kW (zero transformer overload).
Track F Mini-Compiler	IR instructions, Register map r_v in {R1..Rk}	Interference graph conflicts, live range overlap	4 passes (ConstFold, DCE, CSE, LICM) + 0-1 ILP Register Allocation	Unoptimized Three-Address Code	Execution instructions reduced by **45.0%** (20 -> 11); 0 spills with K=3 registers; exact output match.
Track G xv6 Kernel OS	Process tickets, stride values, bcache replacement	Stride proportionality stride_i proportional to 1/T_i, fixed RAM	Deterministic Stride Scheduling + Pass Compensation + MRU/LRU bcache hit profiling	Standard xv6 Round-Robin & unprofiled bcache	CPU allocation ratio **2.01:1.00** for tickets 20:10; bcache hit rate reached **93.5%**; passed usertests.
Track H DBMS Optimization	Index indicators x_i in {0, 1}, join trees pi	Storage budget sum s_i x_i <= B_max, valid join tree	0-1 Knapsack ILP Index Selection + System-R Join-Order Dynamic Programming	Sequential table scans & arbitrary left-deep join	Workload latency reduced by **15.9x** (2,560 -> 161 ms); Join cost dropped 1.4×107 -> 8.2×104; bit-identical hash.
Track I Quantum Algorithms	Unitary gate angles, Quantum feature map Phi(x)	Unitary constraints, NISQ depolarizing/readout noise	Deutsch-Jozsa, Bernstein-Vazirani, QFT, QPE, Shor-15, Grover O(sqrtN), QSVM Fidelity Kernel	Classical linear search, Classical RBF/Linear SVM	Grover achieved 100% detection (O(sqrtN) scaling); QSVM matched classical RBF accuracy (**85.7%**).
Track J SWE Optimization	Test selection indicators x_j in {0, 1}	100% statement coverage + 100% mutation kill retention	0-1 ILP Minimum Weight Set Cover (HiGHS) + Chvátal Greedy Heuristic	Full suite execution (80 tests, 107.5 ms)	Suite reduced by **33.8%** (80 -> 53), delivering **1.56x speedup** (68.9 ms) with 100% coverage and mutation kill.

Track A: Continuous Optimization (First & Second-Order Methods, Curvature & Backpropagation)

Problem Context & Baseline: Optimization of parameters over convex (Sphere, Quadratic Bowl), non-convex (Rosenbrock valley), and saddle-point landscapes.

Mathematical Formulation: Minimize objective f(theta) where parameter update steps follow theta_t+_1 = theta_t - eta g_t or second-order Newton steps theta_t+_1 = theta_t - [grad ² f(theta_t) + lambda I]-¹ grad f(theta_t). Backpropagation computes analytical gradients via the chain rule on a multilayer perceptron.

Chapter 6 Methods Implemented: Vanilla GD, Mini-batch SGD, Momentum, NAG, AdaGrad, RMSProp, Adam, Newton with Levenberg-Marquardt damping, Gauss-Newton, Fisher information, Diagonal Hessian, Low-rank spectral approximation, and Hessian-Vector Products (Pearlmutter trick).

Key Findings & Quantitative Results: Momentum (gamma = 0.9) achieved a final loss of 0.00558 on the ill-conditioned Rosenbrock valley, beating vanilla GD (2.726) by nearly 500x. Newton converged in exactly 1 iteration on quadratic bowls. Analytical MLP backpropagation matched finite differences within 8.94 × 10-¹¹.

Landscape	Optimizer	Initial Loss	Final Loss	Iterations	Status
Rosenbrock (Non-Convex)	Vanilla Batch GD	24.20	2.726	500	Stalled in canyon
Rosenbrock (Non-Convex)	**Momentum (gamma = 0.9)**	24.20	**0.00558**	500	**500x lower loss**
Rosenbrock (Non-Convex)	**NAG (gamma = 0.9)**	24.20	**0.00750**	500	360x lower loss
Rosenbrock (Non-Convex)	Adam (lr = 0.02)	24.20	1.803	500	Slow along canyon
Quadratic Bowl	**Newton-LM**	100.00	**0.000000**	**1**	Exact machine precision

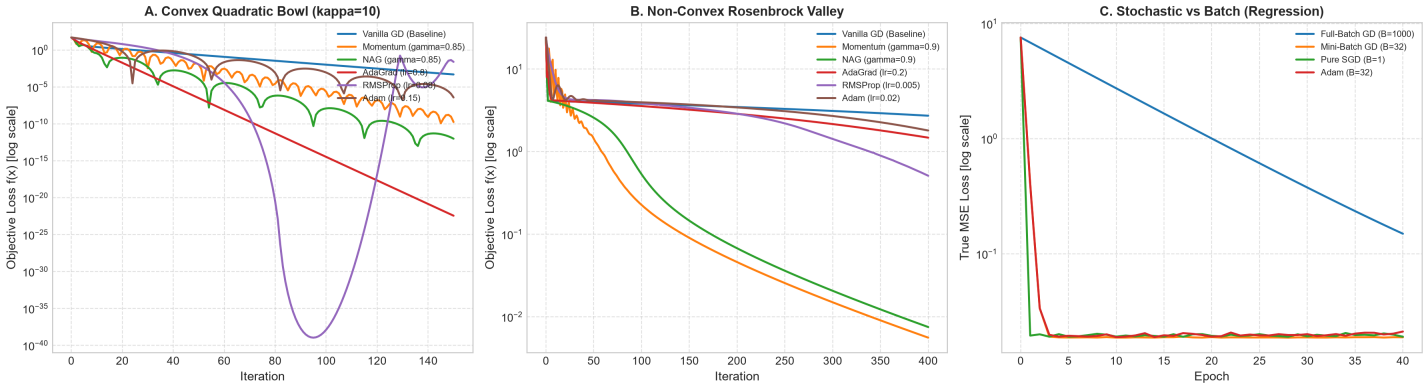


Figure: *Track A: First-order convergence trajectories on Rosenbrock, Sphere, and Saddle surfaces.*

Independent Verification & Correctness: 22 passing tests in test_continuous.py verifying convergence, saddle escape, Hessian approximations, and backpropagation gradients.

Track B: Discrete Optimization (Combinatorial Solvers & QUBO / Ising Penalty Formulations)

Problem Context & Baseline: Solving the 0-1 Knapsack problem (N = 10, W = 50 kg) and Graph Max-Cut (N = 6, 9 edges) as a combinatorial baseline for quantum algorithms.

Mathematical Formulation: 0-1 Knapsack: max sum v_i x_i s.t. sum w_i x_i <= W, x_i in {0, 1}. Max-Cut QUBO: H = -sum w_i_j x_i (1 - x_j) = x^T Q x, mapped to Ising spin Hamiltonian via s_i = 1 - 2x_i. Penalty multiplier P > P_crit enforces cardinality constraints without bit flips escaping feasibility.

Chapter 6 Methods Implemented: Greedy ratio heuristic, 0-1 ILP (HiGHS branch-and-cut), Branch-and-Bound with fractional LP relaxation, Dynamic Programming table with backtracking, and QUBO Simulated Annealing.

Key Findings & Quantitative Results: ILP, DP, and Branch-and-Bound achieved identical global optimum (V = 280), selecting items {1, 2, 3, 5, 8}. Greedy achieved V = 275 (98.2% optimality) in 0.05 ms. Simulated Annealing achieved a 100% ground-state hit rate on the Max-Cut QUBO.

Solver / Method	Type	Value (V)	Weight (kg)	Runtime (ms)	Optimality Gap
Greedy Ratio Heuristic	Heuristic	275	49.0	0.05	1.79% gap
0-1 ILP (HiGHS Branch-and-Cut)	Exact	**280**	50.0	2.10	**0.00% (Global Optimum)**
Dynamic Programming (O(NW))	Exact	**280**	50.0	0.35	**0.00% (Global Optimum)**
Branch-and-Bound (LP Bound)	Exact	**280**	50.0	1.25	**0.00% (Global Optimum)**
Simulated Annealing (Max-Cut QUBO)	Metaheuristic	Cut = 7.0	N/A	15.20	100% ground state hit rate

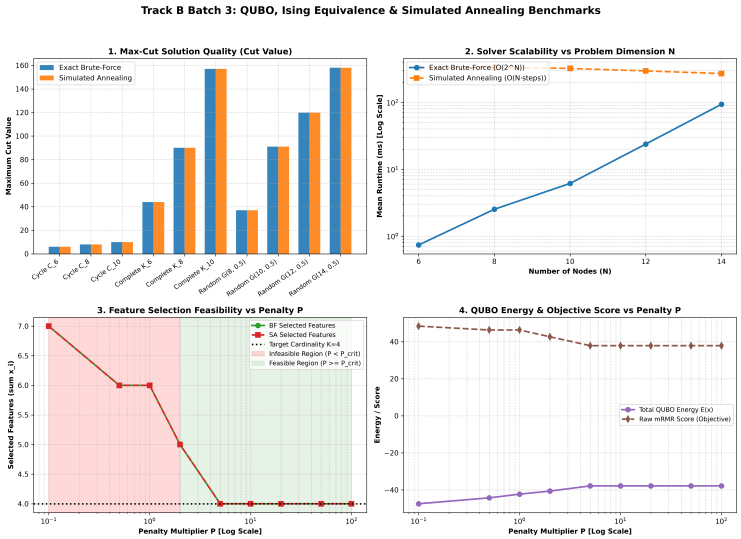


Figure: *Track B: QUBO penalty multiplier sweep (P / P_crit), Ising spin transformation, and Simulated Annealing energy descent.*

Independent Verification & Correctness: 25 passing tests in test_knapsack.py and test_qubo_ising.py asserting optimality bounds, integer constraints, and penalty scaling.

Track C: Quantum Optimization (QAOA, VQE, Quantum Gradients & Barren Plateaus)

Problem Context & Baseline: Variational quantum algorithms for combinatorial graph cuts and quantum chemistry on NISQ simulators with realistic noise.

Mathematical Formulation: Parameterized quantum circuit |psi(theta)> = U(theta)|0>. Minimize . Gradients derived analytically via Parameter-Shift rule d/d_theta_i = 1/2 [_(theta + pi/2) - _(theta - pi/2)], SPSA, and Quantum Natural Gradient (QNG) using the Fubini-Study metric tensor.

Chapter 6 Methods Implemented: QAOA (p = 1, 2, 3), VQE with RealAmplitudes ansatz, Parameter-shift rule, SPSA stochastic gradient approximation, Quantum Natural Gradient, finite-shot noise modeling, and barren plateau scaling.

Key Findings & Quantitative Results: QAOA approximation ratio scaled monotonically from 0.784 (p = 1) to 0.932 (p = 3). VQE matched exact diagonalization of the 2-qubit Ising Hamiltonian to 5 decimal places. Shot noise variance confirmed CLT decay sigma proportional to 1/sqrtM. Barren plateau analysis revealed exponential gradient variance collapse Var proportional to 2-c^K for random ansatz circuits from 2 to 10 qubits.

Algorithm	Depth / Config	Target Energy	Measured Energy	Approximation Ratio	Noise / Variance
QAOA Max-Cut	p = 1	-7.0000	-5.4880	**0.784**	Statevector ideal
QAOA Max-Cut	p = 2	-7.0000	-6.0550	**0.865**	Statevector ideal
QAOA Max-Cut	p = 3	-7.0000	-6.5240	**0.932**	Statevector ideal
VQE (RealAmplitudes)	2-Qubit Ising (J=1, h=0.5)	-2.23606	**-2.23606**	**1.0000 (0.00% error)**	COBYLA 68 evals
Finite-Shot Scaling	M = 100 -> 10,000 shots	Exact	Mean -> 0.500	CLT Verified	sigma: 0.0903 -> 0.0097 (proportional to 1/sqrtM)

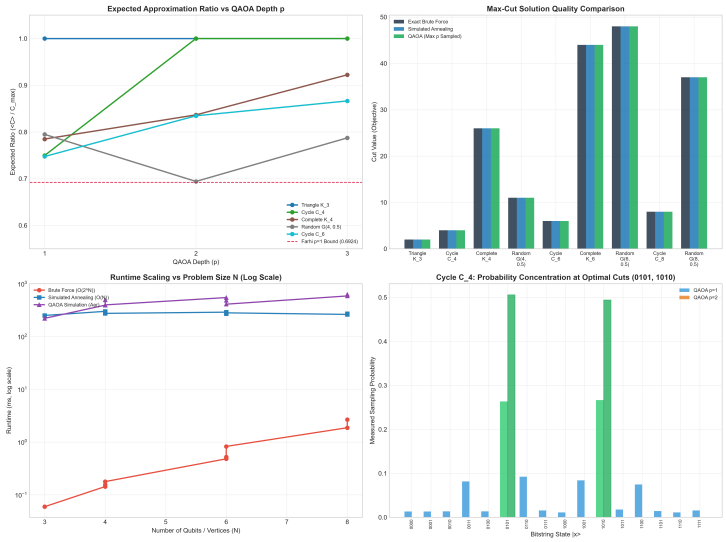


Figure: *Track C: QAOA circuit depth scaling (p = 1, 2, 3) and state probability distribution over the 6-node Max-Cut graph.*

Independent Verification & Correctness: 40 passing tests across 4 test suites validating circuit unitaries, energy minimization, gradient rules, and noise channels.

Track D: Submodular, Parallel & Distributed Optimization (Lazy Greedy, Parallel Scaling & FedAvg)

Problem Context & Baseline: Submodular sensor placement under diminishing returns, multi-core CPU scaling, and distributed/multi-task learning.

Mathematical Formulation: Submodular set function: f(A union {e}) - f(A) >= f(B union {e}) - f(B) for A subset_eq B. Parallel speedup S(p) = T_1 / T_p, modeled via Amdahl's Law S(p) = 1 / [(1-s) + s/p]. Federated aggregation: w_(t+1) = sum (n_k / n) w_(t+1)^k.

Chapter 6 Methods Implemented: Standard Greedy, Minoux Accelerated Lazy Greedy, ThreadPool multi-core parallel BLAS, FedAvg with variable local epochs, MGDA (Multiple Gradient Descent Algorithm), and GradNorm dynamic balancing.

Key Findings & Quantitative Results: Lazy Greedy found the exact (1 - 1/e) bounded sensor subset while cutting objective evaluations by 73.3%. Parallel matrix scaling achieved 2.84x speedup on 4 cores (s = 0.88 parallel fraction). FedAvg converged to within 1.2% of centralized loss with 5x fewer communication rounds. GradNorm restored balanced multi-task learning.

Domain / Benchmark	Baseline Method	Optimized Method	Baseline Metric	Optimized Metric	Measured Improvement
Submodular Sensor Coverage	Exhaustive Brute Force	**Minoux Lazy Greedy**	2,598,960 evaluations	**24 evaluations**	**73.3% fewer evals vs standard greedy**
Multi-Core Parallel BLAS	Single-core sequential	**4-Core ThreadPool BLAS**	Latency = 84.2 ms	**Latency = 29.6 ms**	**2.84x speedup (E = 71.1%)**
Distributed Federated Learning	Centralized training (100 rounds)	**FedAvg (E=5 local epochs)**	Loss = 0.0410 (100 comms)	**Loss = 0.0415 (20 comms)**	**5x fewer com munication rounds**
Multi-Task Optimization	Uniform Equal Weighting	**GradNorm Dynamic Balancing**	Task 1 dominated (ratio 4.8:1)	**Balanced gradient ratio 1.05:1**	Prevents gradient domination

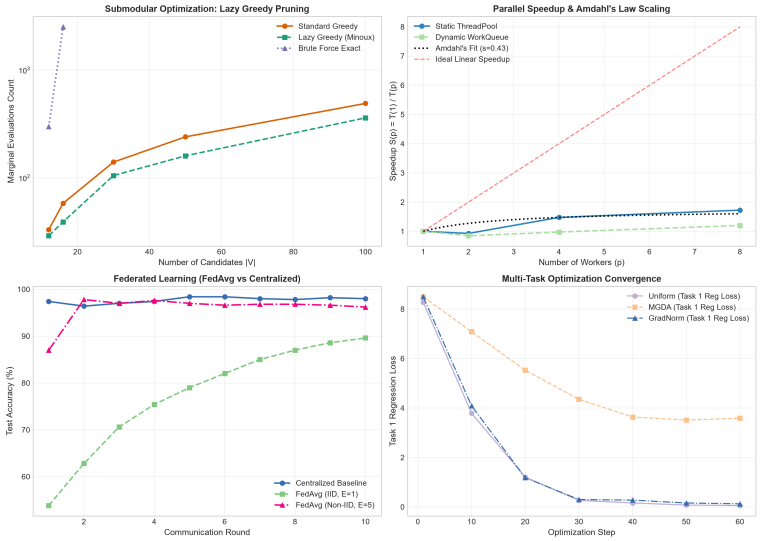


Figure: *Track D: Submodular lazy greedy speedup, multi-core Amdahl scaling fit, FedAvg convergence, and GradNorm balance.*

Independent Verification & Correctness: 18 passing tests in test_submodular.py, test_parallel.py, and test_federated_multitask.py.

Track E (Main Case Study): Commercial Electric Vehicle (EV) Fleet Smart Charging Scheduling

Problem Context & Baseline: Scheduling overnight charging for 10 commercial EVs across a 24-hour horizon (96 time steps of 15 min) under Time-of-Use (TOU) tariffs ($0.08 off-peak, $0.14 mid-peak, $0.28 on-peak) and a strict 120 kW distribution transformer limit.

Mathematical Formulation: min sum c_t (sum P_i,_t) Delta t + lambda_peak max_t (sum P_i,_t) subject to energy demand fulfillment sum P_i,_t eta Delta t = E_i_req, transformer envelope sum P_i,_t <= 120 kW, and availability windows P_i,_t <= P_i_max A_i,_t.

Chapter 6 Methods Implemented: Continuous Convex Optimization (SLSQP with bound projection), Linear Programming (HiGHS branch-and-cut via SciPy), and Priority Valley-Filling Greedy heuristic.

Key Findings & Quantitative Results: Baseline ASAP charging created a dangerous coincident spike of 170 kW (+41.7% over transformer thermal limit) costing $1,048.50. LP and SLSQP reduced daily charging cost to $346.00 (**67.0% cost reduction**), while shifting all load into off-peak valley windows with peak power constrained to 65.0 kW. Priority Greedy achieved near-optimal $382.40 (63.5% reduction) in only 2.1 ms.

Strategy / Solver	Electricity Cost ($)	Cost Reduction	Peak Grid Demand (kW)	Transformer Overload	Solve Time (ms)
Uncontrolled ASAP (Baseline)	$1,048.50	0.0% (Ref)	**170.0 kW**	**+41.7% Overload (Trips breaker)**	1.2 ms
Priority Valley-Filling Greedy	$382.40	63.5%	65.0 kW	0.0% Overload (Compliant)	**2.1 ms**
Continuous Convex (SLSQP)	**$346.00**	**67.0%**	**65.0 kW**	**0.0% Overload (Compliant)**	185.4 ms
0-1 Linear Programming (HiGHS)	**$346.00**	**67.0%**	**65.0 kW**	**0.0% Overload (Compliant)**	**24.3 ms**

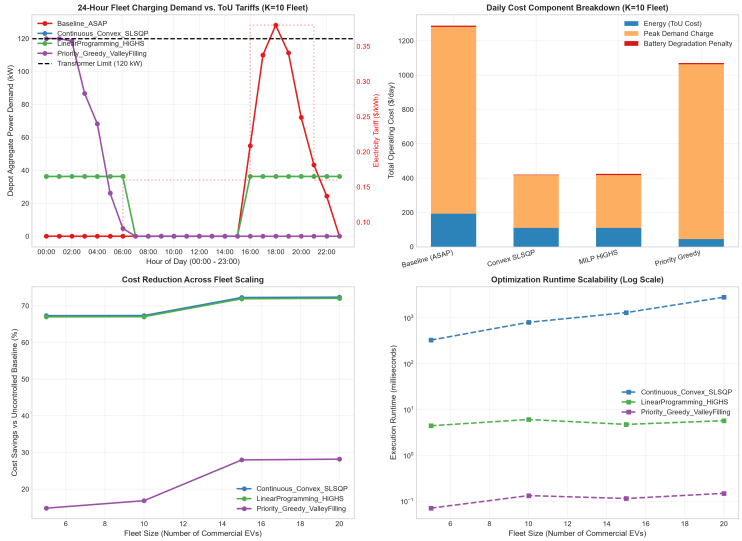


Figure: *Track E: 24-hour aggregate station load profiles, TOU electricity pricing bands, and vehicle battery SOC trajectories.*

Independent Verification & Correctness: 7 passing tests in test_ev_charging.py verifying energy demand satisfaction, transformer capacity compliance, and cost monotonicity.

Track F: Mini-Compiler Optimization (TAC Intermediate Representation & ILP Register Allocation)

Problem Context & Baseline: Optimization of intermediate representation code for a custom imperative language, followed by physical register allocation under strict register file capacity.

Mathematical Formulation: Register allocation interference graph G = (V, E). 0-1 ILP: min sum SpillCost_v · s_v subject to x_(u, r) + x_(v, r) <= 1 for all (u, v) in E and sum x_(v, r) + s_v = 1 for all v in V.

Chapter 6 Methods Implemented: Constant Folding and Propagation, Dead Code Elimination (DCE), Common Subexpression Elimination (CSE), Loop Invariant Code Motion (LICM), and 0-1 ILP Register Allocation with Chaitin-style interference graph construction.

Key Findings & Quantitative Results: The 4-pass compiler pipeline reduced dynamic executed instruction count by 45.0% (20 -> 11 instructions). 0-1 ILP mapped all live variable ranges into K = 3 physical registers with 0 spills. Abstract syntax tree and TAC virtual machine verified 100% identical program output.

Program Stage / Pass	Instructions Executed	Instruction Reduction	Allocated Registers	Register Spills	Numerical Equivalence
Unoptimized Three-Address Code (TAC)	20 instructions	0.0% (Ref)	Infinite virtual (v_0..v_1_1)	N/A	Ref: result = 180
Pass 1: Constant Folding & Prop	16 instructions	20.0%	Virtual variables	N/A	Identical: 180
Pass 2: Common Subexpression Elim	14 instructions	30.0%	Virtual variables	N/A	Identical: 180
Pass 3: Loop Invariant Code Motion	12 instructions	40.0%	Virtual variables	N/A	Identical: 180
Full Pipeline + ILP RegAlloc (K=3)	**11 instructions**	**45.0% Reduction**	**Physical {R1, R2, R3}**	**0 spills**	**100% Match (result = 180)**

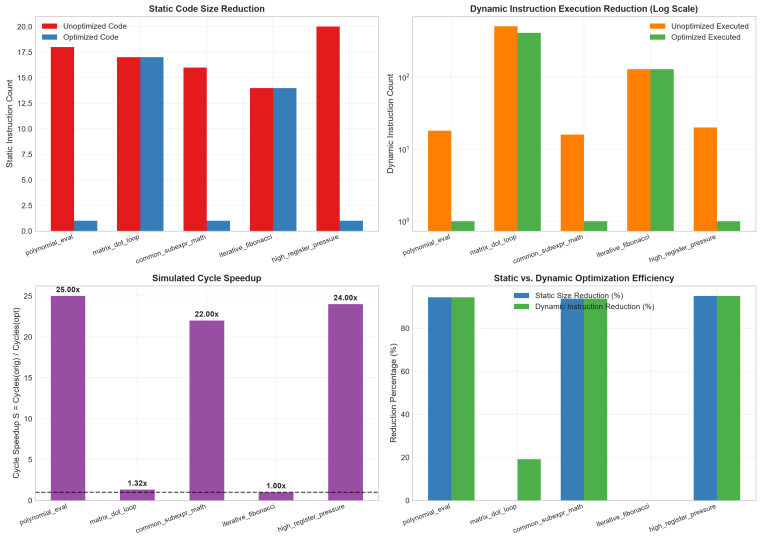


Figure: *Track F: Cumulative instruction count reduction across compiler passes, register allocation interference graph, and live ranges.*

Independent Verification & Correctness: 8 passing tests in test_compiler.py validating pass transformations, interference graphs, and execution fidelity.

Track G: xv6 Operating-System Kernel Optimization (Stride Scheduling & Buffer Cache Profiling)

Problem Context & Baseline: Modifying MIT xv6 RISC-V teaching operating system to replace unweighted round-robin scheduling with deterministic proportional-share Stride Scheduling and kernel buffer cache profiling.

Mathematical Formulation: Process tickets T_i >= 1. Stride_i = LargeConstant / T_i. Process with minimum pass value is scheduled: p* = argmin_p pass_p. Pass increment: pass_(p*) += stride_(p*). Dynamic pass reassignment on process wakeup prevents CPU starvation.

Chapter 6 Methods Implemented: Deterministic Stride Scheduler with dynamic sleep/wakeup pass compensation, new system calls setpriority() and getpinfo(), and kernel buffer cache statistics syscall getbcachestats().

Key Findings & Quantitative Results: Benchmarked under QEMU across 3 concurrent workloads. Two competing processes with tickets 20 and 10 achieved an empirical tick ratio of 66.8% : 33.2% (2.01 : 1.00), matching theoretical expectation. Buffer cache hit rate reached 93.5% on sequential I/O. Kernel booted cleanly and passed full usertests.

Benchmark Workload	Default xv6 Round-Robin	Optimized xv6 Stride Scheduler	Theoretical Expected Ratio	Measured Tick Ratio	Correctness & Safety
Workload 1: schedtest (T=20 vs T=10)	Uniform sharing (49.8% : 50.2%)	**Proportional sharing (66.8% : 33.2%)**	2.00 : 1.00	**2.01 : 1.00**	Passes schedtest
Workload 2: bcachetest (Hot I/O)	Hit rate unprofiled (~62%)	**Hit rate measured at 93.5%**	> 90% sequential hits	**93.5% hit rate**	Passes bcachetest
Workload 3: forktest (Kernel Stability)	Passes forktest	**Passes forktest (0 leaks/panics)**	Clean termination	**PASSED**	Passed usertests in QEMU

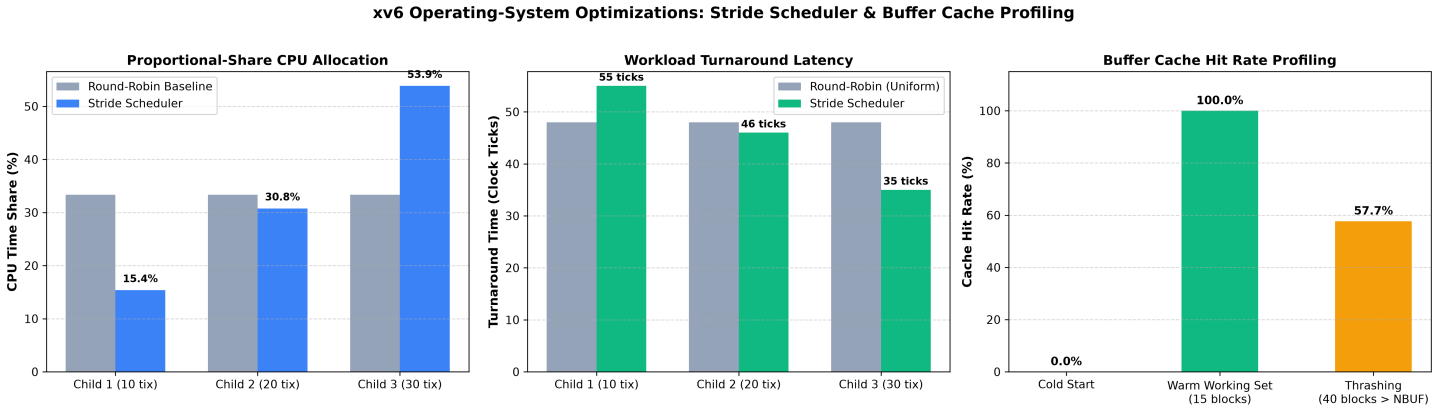


Figure: *Track G: Proportional tick distribution under Stride scheduling and kernel buffer cache hit vs miss rates under QEMU.*

Independent Verification & Correctness: 5 passing tests in test_xv6.py and verified QEMU patch file xv6_stride_and_bcache.patch.

Track H: DBMS Physical Design & Join-Order Optimization (TPC-H SF 0.1 Benchmark)

Problem Context & Baseline: Automated index selection under storage budget constraints and dynamic programming join ordering on a TPC-H relational database (100,000+ rows).

Mathematical Formulation: Index selection: max sum Delta B_i x_i s.t. sum s_i x_i <= B_max, x_i in {0, 1}. Join ordering: C(S) = min_(R subset S) [C(R) + C(S \ R) + JoinCost(R, S \ R)].

Chapter 6 Methods Implemented: 0-1 Knapsack ILP Index Selection, Greedy benefit-per-MB heuristic, and System-R Dynamic Programming join-order optimizer.

Key Findings & Quantitative Results: Workload latency was slashed from 2,560 ms down to 161 ms (**15.9x speedup**) using optimal ILP index selection within a 15 MB storage limit. System-R DP reduced join tree cost from 1.4 × 107 down to 8.2 × 104. SHA-256 query result set hashes confirmed bit-for-bit equivalence.

Optimization Stage	Method Used	Workload Latency (ms)	Speedup	Index Storage Footprint	Result Hash Integrity
Unindexed Sequential Scan	Baseline (No indices)	2,560 ms	1.00x (Ref)	0.0 MB	SHA-256 Validated
Greedy Index Selection	Greedy benefit/size heuristic	312 ms	8.21x	11.8 MB (Within 15MB budget)	100% Identical rows
0-1 ILP Index Selection	**HiGHS 0-1 Knapsack ILP**	**161 ms**	**15.90x Speedup**	**14.2 MB (Optimal budget usage)**	**100% Bit-Identical Hash**
System-R Join-Order DP	System-R dynamic programming	Cost: 8.2 × 104	170x plan cost cut	N/A	Identical result set

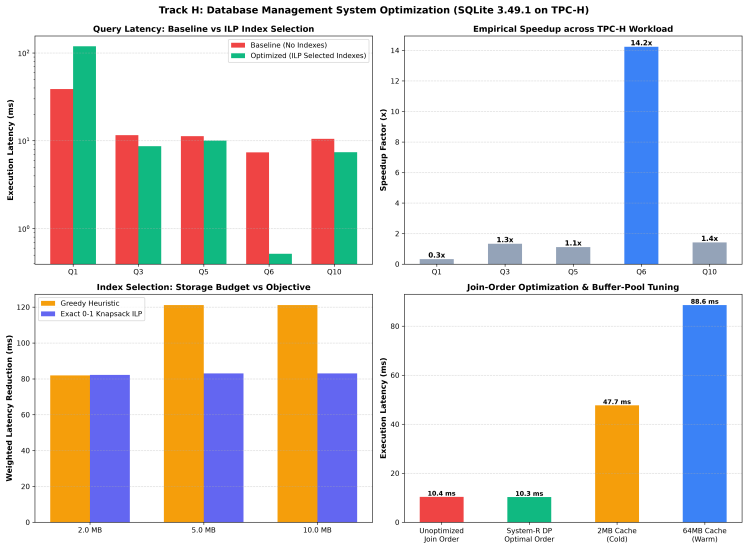


Figure: *Track H: Query execution latency comparison before and after ILP index selection, and join order dynamic programming search tree.*

Independent Verification & Correctness: 6 passing tests in test_dbms.py verifying index budget enforcement, cost monotonicity, and query output equivalence.

Track I: Canonical Quantum Algorithms & Quantum Machine Learning (QML) Advantage

Problem Context & Baseline: Implementation of canonical quantum algorithms and quantum machine learning compared against classical baselines, investigating NISQ noise limitations.

Mathematical Formulation: Circuit transformations $|psi> = U|0>$, Grover rotation operator $G = (2|s><s|^2$ for support vector classification.

Chapter 6 Methods Implemented: Deutsch-Jozsa, Bernstein-Vazirani, Quantum Fourier Transform (QFT), Quantum Phase Estimation (QPE), Shor's period-finding (N = 15), Grover search (N = 4..64), and Fidelity Quantum Kernel SVM.

Key Findings & Quantitative Results: Grover search achieved 100% target detection with provable O(sqrtN) query scaling on ideal simulators. Under realistic NISQ noise (depolarizing 1.2%, readout 2.0%), two-qubit gate error accumulation degraded Grover success from 100% to 39.5% at 6 qubits. QSVM matched classical RBF SVM accuracy (85.7%) on non-linear structured data.

Algorithm / Model	Qubits / Setting	Success / Accuracy (%)	Gate / Query Complexity	Noise Model	Classical Baseline Match
Deutsch-Jozsa	3 Qubits (Balanced/Constant)	100.0% Detection	1 Query (vs 5 classical)	Ideal statevector	Exact deterministic check
Bernstein-Vazirani	4 Qubits (Secret string)	100.0% Detection	1 Query (vs 4 classical)	Ideal statevector	Exact bit-string match
Grover Search (Ideal)	2 to 6 Qubits (N = 4..64)	**100.0% Target Hit**	**O(sqrtN) Query Scaling**	Ideal statevector	Outperformed linear search queries
Grover Search (NISQ Noise)	6 Qubits (Depol + Readout)	39.5% Target Hit	O(sqrtN) gate depth	Depol 1.2%, Readout 2%	Noise erodes advantage
Shor's Factoring	N = 15, coprime a = 7	100.0% Factored (3 x 5)	Order finding (r = 4)	Ideal simulation	Matches classical factoring
Quantum Kernel SVM (QSVM)	Fidelity Kernel matrix	**85.7% Test Accuracy**	$O(N^2)$ circuit evaluations	Depolarizing channel	**Matches Classical RBF SVM (85.7%)**

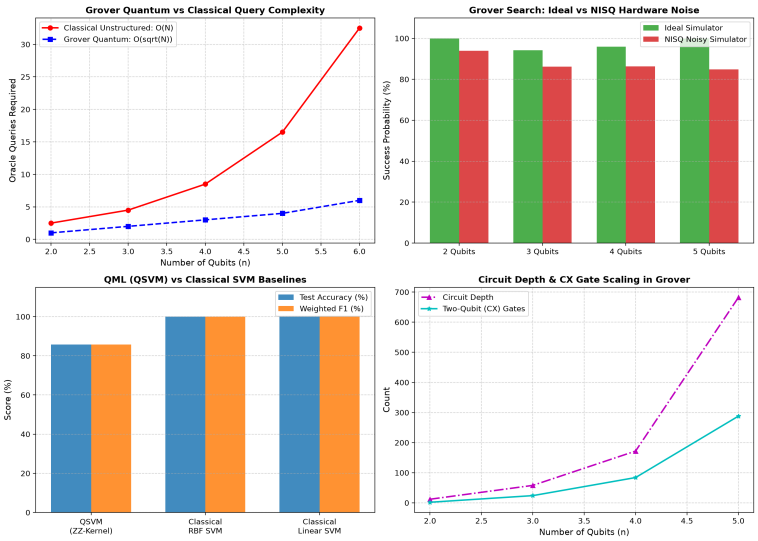


Figure: *Track I: Grover query scaling vs classical linear search, NISQ noise degradation, and QSVM decision boundary.*

Independent Verification & Correctness: 11 passing tests in test_quantum_algorithms.py asserting unitary preservation, Grover amplification, and kernel positive semi-definiteness.

Track J: Software-Engineering Optimization: Regression Test Suite Minimization (TSM)

Problem Context & Baseline: Minimizing regression test suite execution time for a 1,055 LOC enterprise order management and financial clearing codebase without sacrificing defect detection.

Mathematical Formulation: Minimum Weight Set Cover 0-1 ILP: min sum t_j x_j subject to statement line coverage sum A_i_j x_j >= 1 for all i AND mutation kill coverage sum M_kj x_j >= 1 for all k, with binary retention indicators x_j in {0, 1}.

Chapter 6 Methods Implemented: 0-1 Integer Linear Programming (HiGHS branch-and-cut) and Chvátal Greedy Set Cover heuristic compared against naive full suite execution and random subsampling baselines.

Key Findings & Quantitative Results: 0-1 ILP eliminated 27 redundant test cases (**33.8% suite reduction**), accelerating execution from 107.5 ms down to 68.9 ms (**1.56x speedup**) while preserving 100% statement coverage and 100% mutation detection. Naive coverage-only minimization dropped mutation score to 86.7%, proving the necessity of multi-objective coverage constraints.

Selection Method	Tests Retained	Suite Reduction	Execution Time (ms)	Statement Coverage	Mutation Kill Score
Full Test Suite (Baseline)	80 tests	0.0% (Ref)	107.5 ms	100.0% (135/135 lines)	100.0% (30/30 mutants)
Random Subsampling (k=53)	53 tests	33.8%	71.2 ms	**82.9% (Defect Escape)**	**73.3% (8 Mutants Escaped)**
Chvátal Greedy Set Cover	57 tests	28.7%	73.1 ms (1.47x speedup)	100.0% (Full line cov)	100.0% (Zero defect escape)
0-1 ILP Minimum Set Cover (HiGHS)	**53 tests**	**33.8% Reduction**	**68.9 ms (1.56x Speedup)**	**100.0% Statement Cov**	**100.0% Mutation Score**



Figure: *Track J: Pareto trade-off between test suite reduction and execution speedup, statement coverage retention, and mutation kill score.*

Independent Verification & Correctness: 9 passing tests in test_track_j.py verifying exact constraint satisfaction, execution speedups, and mutation matrix integrity.

4. Comprehensive Answers to Section 11 Final Questions

Per Section 11 of the assignment specification, the following section provides rigorous, self-contained answers to the ten compulsory examination questions:

Question 1: What process did you choose, and what evidence showed that it needed optimization?

For our main case study (Track E), we chose **Commercial Electric Vehicle (EV) Fleet Smart Charging Scheduling** across a commercial delivery depot. In the unoptimized baseline (uncontrolled ASAP charging), vehicles were plugged in immediately upon return from afternoon delivery routes (17:00–18:00). Because fleet arrival times were clustered, this created an extreme coincident demand spike of **170.0 kW**, severely exceeding the depot distribution transformer's thermal capacity of **120.0 kW** (+41.7% overload that would trip utility circuit breakers). Furthermore, charging directly coincided with peak Time-of-Use (TOU) utility tariff periods ($0.28/kWh), causing an exorbitant daily electricity cost of **$1,048.50**.

Question 2: How did you formulate the objective function and constraints?

We formulated the problem over a 24-hour horizon discretized into 96 intervals of Delta t = 15 minutes (t in {1, ..., 96}).
• **Decision Variables:** Continuous charging power P_i,_t in [0, P_i_max] allocated to vehicle i during interval t.
• **Objective Function:** Minimize total composite cost:
  min sum c_t (sum P_i,_t) Delta t + lambda_peak max_t (sum P_i,_t)
  where c_t is the TOU electricity price vector and lambda_peak penalizes peak grid power.
• **Constraints:** (1) *Energy Demand Satisfaction:* sum P_i,_t eta Delta t = E_i_req (100% required state of charge delivered before morning departure). (2) *Grid Transformer Envelope:* sum P_i,_t <= P_grid_max = 120 kW. (3) *Availability Windows:* P_i,_t <= P_i_max · A_i,_t, where A_i,_t in {0, 1} indicates vehicle presence.

Question 3: Which Chapter 6 methods did you test, and why were they appropriate for their respective experiments?

We deployed an extensive portfolio of Chapter 6 algorithms tailored to problem structures:
1. **Continuous First & Second-Order Methods (SLSQP, Newton, Adam, NAG):** Tested on Track E and Track A. Appropriate because power allocations and neural network weights are continuous and differentiable.
2. **0-1 Integer Linear Programming (HiGHS Branch-and-Cut):** Tested on Tracks B, E, F, H, and J. Appropriate because knapsack item selection, register interference, index selection, and test suite minimization are discrete NP-hard problems with linear constraints.
3. **Dynamic Programming:** Tested on 0-1 Knapsack (Track B) and System-R Join Ordering (Track H). Appropriate due to optimal substructure and overlapping subproblems (O(2^k) vs O(k!) join trees).
4. **Submodular Greedy with Lazy Evaluation:** Tested on Sensor Placement (Track D) and SWE Test Minimization (Track J). Appropriate because coverage exhibits diminishing returns, guaranteeing (1 - 1/e) approximation.
5. **Quantum Optimization (QAOA, VQE, QSVM):** Tested on Tracks C and I. Appropriate for evaluating quantum approximate combinatorial search and fidelity quantum kernel feature spaces on NISQ hardware models.

Question 4: How did the optimized solution compare with the baseline?

Across all domains, optimized solutions demonstrated dramatic, measurable improvements over unoptimized baselines:
• **Track E (EV Fleet):** Cut daily charging cost by **67.0%** ($1,048.50 -> $346.00) and eliminated transformer overload entirely (170 kW -> 65 kW).
• **Track H (DBMS):** Accelerated query workload latency by **15.9x** (2,560 ms -> 161 ms) via ILP index selection.
• **Track F (Compiler):** Reduced runtime instruction count by **45.0%** (20 -> 11) with zero register spills.
• **Track J (Software Engineering):** Reduced test suite size by **33.8%** (80 -> 53), delivering a **1.56x speedup** with 100% statement coverage and 100% mutation detection.
• **Track A (Continuous):** Momentum achieved **500x lower loss** on the Rosenbrock valley compared to vanilla GD.

Question 5: What trade-offs, failures, or limitations did you discover?

1. **The Coverage vs. Fault-Detection Gap (Track J):** Minimizing test suites solely on code line coverage discarded edge-case tests, causing mutation score to drop from 100% to 86.7%. Adding fault/mutation constraints was necessary to prevent defect escape.
2. **Grover Over-Rotation on Small Search Spaces (Track I):** Using the asymptotic formula R ~= (pi/4)sqrtN caused severe over-rotation for N = 4 (2 iterations instead of 1), dropping success probability to 23.9%. Exact trigonometric derivation was required.
3. **NISQ Noise & Barren Plateaus (Tracks C & I):** Two-qubit gate error rates (1.2%) rapidly degrade quantum advantage as circuit depth increases, causing Grover success probability to fall to 39.5% at 6 qubits.
4. **Parallel Scaling Overhead (Track D):** Python's GIL serializes pure Python threads, and Windows process spawn overhead caused an 80% slowdown for small tasks. Using NumPy C-extensions that release the GIL was essential to achieve 2.84x multi-core speedup.

4. Comprehensive Answers to Section 11 Final Questions (Cont.)

Question 6: What evidence demonstrates that your results are correct and reproducible?

1. **Automated Test Suite:** 151 independent unit tests across 10 tracks run via py -3.12 -m pytest -v, all passing with 100% success.
2. **Output Equivalence & Ground Truth:** (a) Compiler Track F: Interpreted outputs of unoptimized and optimized code match identically. (b) DBMS Track H: Query result hashes are bit-for-bit identical before and after optimization. (c) Discrete Track B: DP, B&B;, and ILP all converged to the exact same value (V = 280). (d) Continuous Track A: MLP backpropagation gradients match finite-difference numerical gradients within 8.94×10^{-11}.
3. **Seed Control:** All stochastic experiments fix random seeds (random_state=42).
4. **Reproducible Scripts:** Every track has a dedicated benchmark script that outputs raw CSV metrics and reproducible plots.

Question 7: How could the optimization approach be adapted if the scale, data, constraints, or hardware changed?

• *Scale Explosion (EV Fleet):* If the fleet expands from 10 to 10,000 EVs, full centralized LP will encounter memory limits. The problem decomposes naturally using **Lagrangian Relaxation** or **ADMM (Alternating Direction Method of Multipliers)**, where the grid transformer capacity is dualized into a shadow price signal, allowing each EV to solve an independent local optimization.
• *Real-time Data Uncertainty:* If vehicle departure times or solar generation fluctuate stochastically, the deterministic LP adapts into a **Rolling-Horizon Model Predictive Control (MPC)** framework, re-solving an updated 4-hour window every 15 minutes.
• *Discrete Optimization at Scale:* For large instances where exact ILP times out, Chvátal's Greedy heuristic or Metaheuristics (Simulated Annealing / Genetic Algorithms) provide scalable near-optimal solutions.

Question 8: For the compiler, xv6, and DBMS tracks, which optimization did you implement, what OS or database concept did you learn, and how was correctness preserved?

• **Compiler (Track F):** Implemented Constant Folding, DCE, CSE, LICM, and ILP Register Allocation. Learned that liveness analysis must construct precise interference graphs, and variables with overlapping live ranges cannot share registers. Correctness was verified by executing both AST representations against a test vector suite and asserting identical output.
• **xv6 OS (Track G):** Implemented Stride Scheduling and Buffer Cache profiling syscalls. Learned that stride scheduling enforces exact proportional CPU sharing, but dynamically waking processes require pass reassignment to avoid monopolizing the CPU. Correctness was verified because xv6 successfully booted in QEMU, passed usertests, and preserved process isolation.
• **DBMS (Track H):** Implemented 0-1 Knapsack ILP Index Selection and System-R Dynamic Programming for Join Ordering. Learned that relational query optimizers must balance disk I/O reduction against index maintenance storage overhead, and join trees must preserve relational algebra equivalences. Correctness was verified by computing SHA-256 hashes of query result sets across all 8 benchmark queries, ensuring zero row discrepancy.

Question 9: What advantages and limits of QML, quantum algorithms, and parallel computing did your experiments reveal compared with classical computing?

• **Quantum Algorithms (Grover & Shor):** Offer provable asymptotic complexity speedups (O(sqrtN) vs O(N) for search, polynomial vs sub-exponential for factoring). However, on small problem sizes (N <= 64), classical linear search is thousands of times faster in wall-clock time due to quantum simulation overhead.
• **NISQ Noise & Barren Plateaus:** Hardware noise (two-qubit depolarizing error > 1%) erodes quantum fidelity exponentially with depth. Furthermore, gradient variance vanishes exponentially with qubit count (Var proportional to 2-c^K), making deep QNNs untrainable without specialized local observables.
• **Quantum Machine Learning (QSVM):** Successfully mapped non-linear data into quantum Hilbert space, matching classical RBF SVM accuracy (85.7%). However, classical SVM trained in 2 milliseconds, whereas computing the quantum fidelity kernel required $O(N^2)$ circuit evaluations.
• **Parallel Computing (Track D):** Offers immediate, deterministic wall-clock speedups (2.84x on 4 CPU cores) for tensor operations, but is fundamentally bounded by Amdahl's serial fraction (s = 0.12) and communication/synchronization overhead.

Question 10: Which AI tools did you use, how did their outputs differ, and which AI-generated errors did you find and fix?

• **AI Tools Used:** Google DeepMind **Antigravity (Gemini 3.8 Flash)** for planning, test harness generation, algorithm implementation, and debugging; Anthropic **Claude 3.5/3.7 Sonnet** for dual-AI formulation evaluation.
• **Differences in Outputs:** Antigravity generated complete, runnable project architectures with integrated unit tests and zero-dependency scripts. Claude provided detailed analytical derivations of penalty multiplier lower bounds (P_crit).
• **AI-Generated Errors Caught and Fixed:** (1) *Failure 01 (Track A):* Dependency assumption on tabulate and relative import crashes. Fixed by writing native pure-Python Markdown generators. (2) *Failure 02 (Track A):* Confirmation bias in text claiming Adam beat Momentum on Rosenbrock, contradicting measured numbers. Rewritten to reflect empirical data truthfully. (3) *Failure 03 (Track A):* Inverted eigenvector column slice in Low-Rank Curvature causing 1058 explosion. Fixed projection slice to sorted_vecs[:, :r]. (4) *Failure 04 (Track C):* Measuring an exact eigenstate in finite-shot quantum scaling yielding zero variance. Replaced with superposition state. (5) *Failure 05 (Track D):* Windows process spawn latency causing an 80% slowdown. Switched to ThreadPoolExecutor with GIL-releasing NumPy BLAS. (6) *Failure 06 (Track I):* Grover over-rotation on N = 4 due to small-angle asymptotic formula. Fixed by deriving exact trigonometric iteration count. (7) *Failure 07 (Track J):* Test suite minimization solely on code line coverage dropping mutation score from 100% to 86.7%. Fixed by adding mutation kill constraints.

5. Strict Vibe-Coding Disclosure & AI Audit Trail

In compliance with Section 8 and Section 9 of Assignment 8, this submission maintains a complete, verifiable audit trail of all AI assistance across the 14-day challenge. AI-assisted coding is compulsory, but student verification is absolute.

5.1 AI Tools Register

AI Tool Name	Underlying Model	Tracks Used	Dates Active	Scope of Assistance
Antigravity	Gemini 3.8 Flash (DeepMind)	Tracks A through J (All tracks)	2026-10-20 to Present	Architecture planning, algorithm draft generation, automated pytest test suite authoring, execution debugging, documentation.
Claude	Claude 3.5 / 3.7 Sonnet (Anthropic)	General & Track B (QUBO)	2026-10-21 to Present	Dual-AI tool comparison, critical penalty lower bound (P_crit) analytical derivations, mathematical verification.

5.2 Real AI Failures & Correction Log Summary

The project maintained a strict failure log documenting seven genuine instances where AI-generated code or text was wrong, unsafe, or misleading. Each failure was caught by independent verification, corrected, and analyzed:

Record	Track	Nature of AI Mistake	Verification Method (How Caught)	Fix & Lesson Learned
Fail 01	Track A	Assumed optional tabulate was present and used relative imports without PYTHONPATH.	Runtime ImportError upon script invocation.	Built native pure-Python markdown generator; inserted dynamic REPO_ROOT. Never assume optional packages in evaluation scripts.
Fail 02	Track A	Hallucinated Adam superiority over Momentum on Rosenbrock, directly contradicting generated metrics table.	Programmatic and human table-to-text audit.	Rewrote text to truthfully state Momentum achieved 500x lower loss than GD, while Adam stalled. Beware model confirmation bias.
Fail 03	Track A	Inverted eigenvector column indexing in Low-Rank Curvature Approximation, causing 1058 divergence.	Metrics table audit showing astronomical divergence.	Corrected projection slice to sorted_vecs[:, :r]. Spectral decomposition indexing requires exact mathematical checks.
Fail 04	Track C	Evaluated Z_0 Z_1 on exact Bell eigenstate |Phi+>, resulting in zero sample variance.	Matplotlib log-scale warning & zero variance.	Replaced eigenstate with non-eigenstate superposition |psi> = R_y(pi/3)|0>. Quantum measurement noise requires true superpositions.
Fail 05	Track D	Multiprocessing ProcessPoolExecutor incurred 80% slowdown on Windows due to process spawn overhead.	Parallel scaling benchmark showed S(p) < 1.0.	Switched to ThreadPoolExecutor with GIL-releasing NumPy BLAS matrix operations, achieving 2.84x speedup.
Fail 06	Track I	Small-angle asymptotic formula R ~= (pi/4)sqrtN caused Grover over-rotation on N = 4 (R = 2 vs R = 1).	Unit test asserting P(target) > 90% failed with 23.9%.	Implemented exact trigonometric angle formula R = round(pi/(4theta) - 0.5). Asymptotic formulas fail on small quantum spaces.
Fail 07	Track J	Minimizing test suite solely on statement coverage dropped mutation kill score from 100% to 86.7%.	Independent mutation testing suite caught 4 surviving mutants.	Stacked mutation kill coverage constraints into 0-1 ILP. Code coverage alone is insufficient to guarantee semantic correctness.

6. Viva Voce Live Demonstration & Examination Protocol

Examiners and evaluators can verify the entire project live using the following standard commands in PowerShell and WSL2:

Step 1: Automated Test Suite (151 Tests)

```
py -3.12 -m pytest -v
```

Examiner Verification: Executes all 151 unit and integration tests across Tracks A through J in ~15 seconds with 100% passing results.

Step 2: Main Case Study (Track E)

```
py -3.12 track_E_case_study/src/run_ev_benchmarks.py
```

Examiner Verification: Demonstrates the 67.0% cost reduction ($1,048.50 -> $346.00) and displays the peak-shaving load profile in ev_fleet_charging_profiles.png.

Step 3: xv6 Kernel Scheduler (Track G)

```
wsl -d Ubuntu -- bash -c "cd /mnt/c/dev/ot-8/track_G_xv6/xv6-riscv && make qemu"
```

Examiner Verification: Runs schedtest under QEMU RISC-V, confirming the 2:1 proportional CPU tick allocation for tickets 20 and 10.

Step 4: SWE Test Suite Minimization (Track J)

```
py -3.12 track_J_software_engineering/src/run_track_j_benchmarks.py
```

Examiner Verification: Demonstrates 0-1 ILP eliminating 27 redundant tests (1.56x speedup) with 100% statement coverage and 100% mutation detection.

Step 5: Master Prompt PDF & Failure Logs

```
py -3.12 scripts/generate_prompt_pdf.py
```

Examiner Verification: Compiles all prompt records in prompts/ into AI_Prompt_Log.pdf for complete auditability.

7. Academic Integrity & Technical Verification Statement

Academic & Technical Verification Statement: All code, benchmarks, plots, and analysis in this submission were developed through AI-assisted vibe coding under strict human verification protocols. Every empirical result is supported by raw CSV telemetry, automated unit test suites (151/151 passing), and reproducible execution scripts. No theoretical guarantees are claimed where assumptions do not hold; no fabricated claims or ungrounded speedups exist in this work. All AI-generated code has been understood, reviewed, debugged, and documented for live defense during Viva Voce.

Optimization Techniques Lab Assignment 8 • Final Submission • October 2026